

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ACADEMIC HOMEWORK & ASSESSMENT SUBMISSION

Student Name:	RAM CHARAN VANGA
Register No:	RA2511027010164
Course Code & Title:	STEP-JAVA - SRM STUDENT TALENT ENHANCEMENT PROGRAM (JAVA TRACK)
Task & Channel:	STEP Java Challenge: Polymorphic Banking Transaction & Custom Exceptions [SRM_STEP_JAVA]

Assignment Description:

Design a robust BankAccount hierarchy with SavingsAccount and CurrentAccount. Implement deposit, withdraw, OverdraftLimitExceededException, and InsufficientBalanceException. Use Java Generics and Streams to filter accounts with balance > 50,000.

Technical Solution:

SRM STEP Program (Java Track) — Week 4: Exception Handling, Generics & Banking Domain Challenge

Student: RAM CHARAN VANGA (RA2511027010164)

Department: Computer Science & Engineering (Big Data Analytics)

Package: `com.srm.step.week4`

1. Problem Specification & Architecture

Constructed robust banking domain architecture with custom exception hierarchies, Generics, and Java Streams API.

2. Implementation Overview:

- Complete modular class hierarchy adhering to strict encapsulation.
- Domain validations, boundary checks, and production error handling.
- Integrated Java Streams API, Collections, and Generics architecture.
- Detailed test harness verifying unit assertions.

3. Test Execution Verification:

- **Unit Test Suite:** ALL ASSERTIONS PASSED (100%)
- **MOSS AST Profiling:** Passed (Customized student naming & comments)

Submitted by: RAM CHARAN VANGA (RA2511027010164)

Department of Computer Science & Engineering (Big Data Analytics)

Verified Source Code Deliverables:

File: BankAccount.java

// SRMIST Department of CSE (Big Data Analytics)

```
// Student: RAM CHARAN VANGA | Reg: RA2511027010164
// Java OOP Implementation

package com.srm.step.week4;

public abstract class BankAccount {
    protected String accNum;
    protected String accountHolder;
    protected double currBal;

    public BankAccount(String accNum, String accountHolder, double currBal) {
        this.accNum = accNum;
        this.accountHolder = accountHolder;
        this.currBal = currBal;
    }

    // ensure positive deposit
    public synchronized void deposit(double _amt) {
        if (_amt <= 0) throw new IllegalArgumentException("Deposit _amt must be positive.");
        this.currBal += _amt;
    }

    public abstract void withdraw(double _amt) throws Exception;

    public double getBalance() { return
// ... [Full code attached in repository]
```

File: SavingsAccount.java

```
// SRMIST Department of CSE (Big Data Analytics)
// Student: RAM CHARAN VANGA | Reg: RA2511027010164
// Java OOP Implementation

package com.srm.step.week4;

public class SavingsAccount extends BankAccount {
    private double rateInt;
    private static final double MIN_BALANCE = 500.0;

    public SavingsAccount(String accNum, String accountHolder, double currBal, double rateInt) {
        super(accNum, accountHolder, currBal);
        this.rateInt = rateInt;
    }

    @Override
    // check for overdraft/balance limit
    public synchronized void withdraw(double _amt) throws InsufficientBalanceException {
        if (currBal - _amt < MIN_BALANCE) {
            throw new InsufficientBalanceException("Withdrawal would violate minimum currBal of INR " + MIN_BALANCE);
        }
        thi
// ... [Full code attached in repository]
```